Implementing Text2SQL using Model Context Protocol (MCP)

Architecture, Diagrams, and Code Strategy

Wook-Shin Han

November 30, 2025

Agenda

1. Overview of MCP Architecture
2. Code Implementation Strategy
3. Process Flow & Security
4. Extension: Mixed Chat & Text2SQL

What is Model Context Protocol (MCP)?

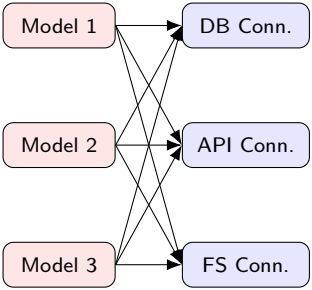
- **Definition**: An open standard that enables AI models to interact with external data and tools securely.
- **Key Concept**: Decouples the **Connectors** from the **AI Models**.
- **Text2SQL Context**: Instead of hard-coding SQL logic into the chatbot, we expose the database as an **MCP Server**.

Why MCP?

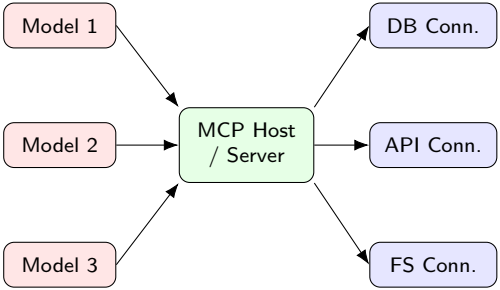
Solves the $N \times M$ integration problem. Write the database connector once, use it with Claude, ChatGPT, or any MCP-compliant client.

The $N \times M$ Integration Problem

Motivation for MCP: Without a shared protocol, each model must integrate separately with each data/tool connector.



Without MCP: $N \times M$ direct integrations



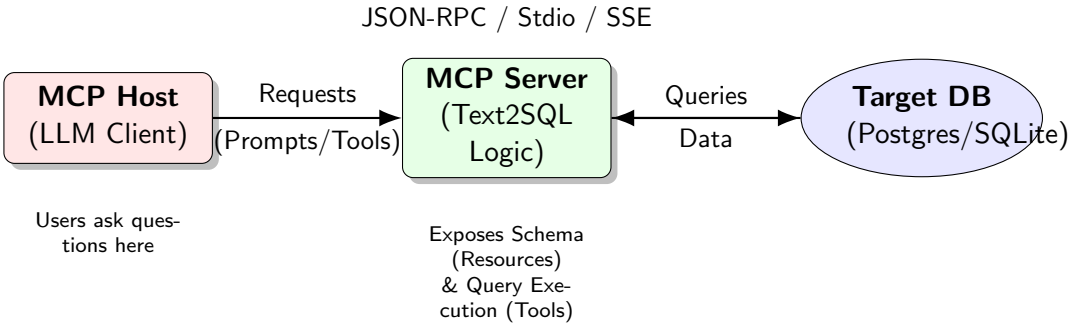
With MCP: $N + M$ integrations via a shared protocol

MCP Entities & Responsibilities

Who does what in MCP?

- **MCP Host**
  - Owns the user interface (chat app, IDE plugin, desktop client, etc.).
  - Talks to both the **LLM API** and one or more **MCP servers**.
  - Decides when to request resources or call tools on a given MCP server.
- **LLM (Model)**
  - Reads the user's question and any context (e.g., schema text).
  - Performs **Text2SQL**: converts natural language into a SQL query.
  - Chooses which tool to call and with which arguments (e.g., SQL string).
  - Turns structured results (JSON rows) back into a natural-language answer.
- **MCP Server**
  - Exposes **resources** (read-only data such as schema, config, files).
  - Exposes **tools** (actions such as `run_read_only_sql`).
  - Connects to external systems (databases, APIs, file systems, etc.).
  - Does **not** call the LLM itself; it simply serves data and executes tools.

System Architecture



High-level Flow:

1. Host fetches schema from the MCP server (resource).
2. Host calls the LLM with: schema + user question.
3. **Inside the LLM: Text ⇒ SQL (Text2SQL)**.
4. Host sends the generated SQL to the MCP tool (`run_read_only_sql`).
5. MCP server executes the SQL on the DB and returns JSON rows.
6. Host asks the LLM to turn rows back into a natural-language answer.

1. Server Initialization

We use the Python MCP SDK (`mcp`). The server instance connects the protocol to our logic.

```python
from mcp.server.fastmcp import FastMCP
import sqlite3

# Initialize the MCP Server
mcp = FastMCP("Text2SQL Service")

# Database Connection Helper
def get_db_connection():
    conn = sqlite3.connect("database.db")
    conn.row_factory = sqlite3.Row
    return conn

```

Note: `FastMCP` handles the underlying JSON-RPC communication automatically.

2. Implementing Resources (Schema Sharing)

Resources allow the LLM (via the host) to "read" the database structure before writing queries.

```python
@mcp.resource("sqlite://schema")
def get_schema() -> str:
    """Returns the DDL (CREATE TABLE statements) of the DB."""
    conn = get_db_connection()
    cursor = conn.cursor()

    # Get all table definitions
    cursor.execute("SELECT sql FROM sqlite_master WHERE type='table'")
    tables = cursor.fetchall()

    schema_str = "Database Schema:\n"
    for table in tables:
        schema_str += table["sql"] + ";\n"

    return schema_str

```

3. Implementing Tools (Execution)

Tools are executable functions. We need a tool to run the generated SQL.

```python
@mcp.tool()
def run_read_only_sql(query: str) -> str:
    """Executes a SELECT query. BLOCKS modification queries."""

    # 1. Security Check (Basic)
    forbidden = ["DROP", "DELETE", "UPDATE", "INSERT", "ALTER"]
    if any(word in query.upper() for word in forbidden):
        return "Error: Only SELECT queries are allowed."

    # 2. Execution
    try:
        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.execute(query)
        results = [dict(row) for row in cursor.fetchall()]
        return str(results)  # Return data as JSON string
    except Exception as e:
        return f"Database Error: {str(e)}"

```

4. Implementing Prompts

Prompts guide the LLM on how to behave. The *Text* → *SQL* conversion happens when the host calls the LLM with this prompt and the database schema.

```
@mcp.prompt()
def text_to_sql_prompt(user_question: str) -> dict:
    return {
        "messages": [
            {
                "role": "user",
                "content": {
                    "type": "text",
                    "text": f"""
You are a SQL expert.
1. Use the resource 'sqlite://schema' to understand the DB.
2. Convert this question to SQL: '{user_question}'
3. Call 'run_read_only_sql' with the result.
"""
                }
            }
        ]
    }

```

5. Host Orchestration (1/2): Schema & LLM Call

Where exactly does Text2SQL happen? On the *host* side, inside the LLM call that uses the schema and user question.

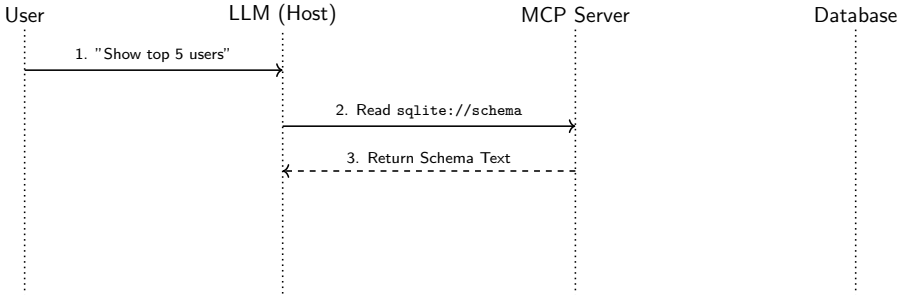
Steps 1–3: Prepare context and call the LLM

```python
def answer_question(user_question: str):
    # 1. Get schema text from MCP server
    schema = mcp_client.read_resource("sqlite://schema")

    # 2. Build messages for the LLM
    messages = [
        {"role": "system",
         "content": "You are a Text2SQL assistant."},
        {"role": "user",
         "content": f"Schema:\n{schema}\n\nQuestion: {user_question}"}
    ]

    # 3. Call the LLM with the tool definition
    result = llm.chat(
        messages=messages,
        tools=[run_read_only_sql_tool_def]
    )
```

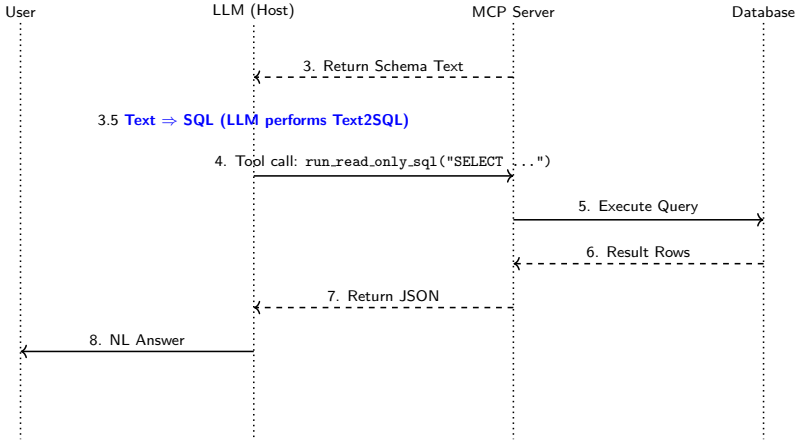
Interaction Flow (Sequence 1/2): Context Gathering



Steps 1–3: Collecting Context

- Step 1: The user sends a natural language question to the host LLM.
- Step 2: The host fetches the database schema from the MCP server using the resource `sqlite://schema`.
- Step 3: The MCP server returns the schema text so the host/LLM can see the table and column structure.

Interaction Flow (Sequence 2/2): Text2SQL & Execution



Text → SQL Timing & Flow

- **Step 3.5:** After seeing the schema and question, the LLM converts text into a SQL query and chooses the tool.
- Step 4: The generated SQL is wrapped as a **tool call** to `run_read_only_sql`.
- Steps 5–7: The MCP server executes the SQL and returns structured rows.
- Step 8: The LLM turns those rows into a natural-language answer for the user.

Security & Best Practices

1. Read-Only Enforcement

- Never use a DB user with write permissions (INSERT/DROP).
- Implement AST parsing (e.g., `sqlparse`) to validate query type before execution.

2. Context Management

- Do not dump the entire schema for large databases.
- Use **RAG for Schema**: Search relevant tables based on the user question before building the context.

3. Error Recovery

- If SQL fails, return the specific error message to the LLM.
- Allow the LLM to self-correct and retry the `run_read_only_sql` tool.

Extending to a Mixed Chat & Text2SQL Assistant

Typical real-world assistant:

- Needs to handle **general chat** (small talk, general Q&A, explanations).
- Needs to handle **data questions** that require SQL over the database.
- Users do *not* want to switch modes manually for every question.

Key design idea (host-side):

- Add an **intent detection / routing layer** in the host:
  - Decide whether the user's message is **"DB-related"** or **"pure chat"**.
  - For DB-related questions, use the full Text2SQL + MCP pipeline.
  - For pure chat, call the LLM **without** Text2SQL tools.
- The MCP server itself stays **unchanged**:
  - Same resources (schema).
  - Same tools (`run_read_only_sql`).

Host-Side Routing: When to Call MCP

Goal: Automatically decide per message:

$$\text{intent} \in \{\texttt{CHAT}, \texttt{TEXT2SQL}\}$$

Common routing strategies (host):

- **LLM-based intent classification**
  - Ask the LLM (with a short system prompt) to label the message:
  - e.g., `"Is this a database question that requires SQL? (yes/no)"`.
  - If `yes` $\Rightarrow$ run Text2SQL flow, else general chat.
- **Heuristics / rules (simple but effective)**
  - Detect words like `SELECT`, `table`, `column`, `top N users`, or explicit mentions of the data source.
  - Track recent turns: if the last few messages used Text2SQL, keep using Text2SQL until the topic clearly changes.
- **User override (optional)**
  - Allow commands like `/sql` or `/chat` to override the router.
  - Useful for debugging and teaching.

Extended Host Orchestration: Chat + Text2SQL

Host-level control flow

```python
def handle_message(user_message: str, history: list):
    """
    Single entry point for the chatbot.
    Decides whether to use Text2SQL or plain chat.
    """
    # 1. Detect intent (CHAT vs TEXT2SQL)
    intent = detect_intent(user_message, history)

    if intent == "TEXT2SQL":
        # 2a. Use the Text2SQL pipeline (with MCP tools)
        return answer_question_via_text2sql(user_message, history)
    else:
        # 2b. Use regular chat (no MCP tools, or a smaller tool set)
        return general_chat(user_message, history)

```

The MCP server does not change; we only add a **router** *and keep two host-side flows: one for general chat and one for Text2SQL.*

Host Helper: generate_chat (Plain Chat)

Question: Does generate_chat call the LLM again inside?
Answer: Yes. It is a thin wrapper around a `llm.chat` call for **normal chat** (no Text2SQL tools).

Implementation sketch (host side):

```python
def generate_chat(user_message: str, history: list) -> str:

    # 1. Build messages: system prompt + history + new user message
    messages = [
        {"role": "system",
         "content": "You are a helpful assistant."}
    ]
    messages.extend(history)  # previous turns
    messages.append({
        "role": "user",
        "content": user_message
    })
    # 2. Single LLM call (no MCP tools)
    result = llm.chat(messages=messages)

    # 3. Update history outside this function if needed
    return result.content

```

What We Extend (and What We Don't)

Host changes (extensions):

- Add **intent detection** to separate CHAT vs TEXT2SQL messages.
- Maintain **conversation state** so follow-up questions can reuse context.
- Decide which tools to expose to the LLM:
  - CHAT: often no tools or a minimal set.
  - TEXT2SQL: expose `run_read_only_sql` and schema resources.
- Optionally, add UI controls:
  - Show generated SQL to the user.
  - Allow switching modes or re-running a query.

MCP server (largely unchanged):

- Same implementation of **resources** (schema) and **tools** (SQL execution).
- No knowledge of "chat", "intent", or "routing".
- Acts as a **stateless connector** to the database.

Conclusion

- **Standardization**: MCP provides a unified way to connect LLMs to SQL databases.
- **Safety**: Execution logic is sandboxed within the server tools.
- **Flexibility**: The same MCP server works with Claude Desktop, IDEs, and custom apps.
- **Clarity**: Text2SQL is a host-side LLM step, expressed as a tool call with a SQL argument.

Thank You!

Q&A